



PROJECT HANDBOOK

# SigniQ

---

Engineering reference: workspace architecture, configuration, the complete API surface, data formats, testing strategy and operator notes.

TEAM PROMETHEUS · SIH26141

Quantum-secured document signatures · classical software simulation · 2026-09-29

# 1. System overview

SigniQ · Team Prometheus · SIH26141 · classical software simulation — no quantum hardware

SigniQ is a Rust workspace (edition 2021, 12 crates) behind an Axum HTTP server that also serves a built React dashboard. Quantum protocol behavior — six-state QKD, teleportation, Bell measurements — is **classical software simulation** throughout; nothing here touches quantum hardware. Security-relevant events append to a hash-chained Merkle audit ledger on disk.

```
frontend (React 19 + Vite) ⇌ server (Axum, REST + SSE) ⇌ crates
```

Three binaries matter: the server ( `cargo run -p server` ), the terminal UI ( `cargo run -p tui` ), and the CLI demo ( `cargo run -p main_app` ).

## 2. Crate map

| Crate     | Responsibility                       | Key items                                                                                                                                                                                                                                                    |
|-----------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| quantum   | Six-state QKD model, channel physics | PauliBasis / PauliState , generate_eigenstates , ChannelSession , relays, decoy states, reconcile / reconcile_adaptive , CHSH ( src/chsh.rs )                                                                                                                |
| pa        | Privacy amplification                | Toeplitz::random / multiply (GF(2)), EntropyBudget , distill / distill_seeded , leftover-hash bound ( lh1_distance_bound )                                                                                                                                   |
| detection | Statistical detection engine         | Hoeffding/Chernoff bounds, exact binomial tails ( src/bounds.rs ), dynamic thresholds, finite_key_ok , eve_information_fraction                                                                                                                              |
| qds       | Quantum digital signatures           | teleport.rs (statevector engine, Born-rule collapse), lib.rs ( Trent , sign , verify , theory_forgeries_probability ), six_state.rs (Weng et al. protocol), attacks.rs , metrics.rs (confusion matrix), temporal.rs (replay trap), consensus.rs (ring)       |
| sealing   | .qsig containers                     | AES-256-GCM payload (v2), sealed envelope (v3), key commitments, Shamir GF(251) quorum                                                                                                                                                                       |
| audit     | Tamper-evident ledger                | AuditEntry , hash chain ( AUD1 leaves), Merkle root (RFC 6962 interior tags), inclusion proofs                                                                                                                                                               |
| attacks   | QKD-channel attack scenarios         | intercept-resend fractions, scenario batteries                                                                                                                                                                                                               |
| crypto    | Shared primitives                    | HMAC, hashing helpers                                                                                                                                                                                                                                        |
| server    | HTTP API + static hosting            | route table (§5), auth ( src/auth.rs ), QDS state + keys ( qds_state.rs , qds_keys.rs ), document API ( doc_api.rs )                                                                                                                                         |
| main_app  | CLI demo binary                      | seeded five-scenario walk: secure QKD → HMAC; intercept-resend → abort; Chernoff–Hoeffding dossiers for both channels; seal → tamper → verify with audit-chain + inclusion-proof checks; full attack-class evaluation sweep (detection / false-accept rates) |

| Crate | Responsibility                  | Key items                                                                                                                                                                             |
|-------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tui   | Terminal live monitor (ratatui) | keys: 1 / 2 / 3 or Tab cycle scenario (secure / attack / mixed 30% intercept) · n / N fiber noise -/+ · r / R relay hops -/+ · s seal · v verify · t tamper · w quorum · q / Esc quit |
| tests | Cross-crate integration         | end-to-end protocol tests                                                                                                                                                             |

### 3. Build, run, test

Prerequisites: Rust (stable, edition 2021) and Node.js 20+ with npm.

```
# Frontend (dashboard must be built before the server can serve it)
cd frontend && npm ci && npm run build && cd ..

# Server - serves the API and the built dashboard
cargo run -p server                # http://127.0.0.1:8080 by default

# Terminal UI / CLI demo
cargo run -p tui
cargo run -p main_app

# Tests
cargo test --workspace            # 21 test binaries
cd frontend && npm run lint        # oxlint
```

**Frontend dev loop:** `npm run dev` (Vite) in `frontend/` while the API server runs; Vite proxies to the API per its config. Production build output lands in `frontend/dist/`, which the server serves (path overridable, §4).

### Documentation PDFs

The four professional documents (README, whitepaper, handbook, manual) convert to branded print-ready PDFs via one repeatable command (Markdown → marked → Chrome headless print-to-PDF, A4, Prometheus-flame cover page):

```
cd frontend && npm install          # one-time (marked is a devDependency)
node ../scripts/build-docs-pdf.mjs
```

Output lands in `target/docs-pdf/*.pdf` (gitignored). Chrome/Edge is located automatically; override with the `CHROME` environment variable. The converter renders GFM tables with repeat headers, keeps code blocks and table rows intact across page breaks, and was verified page-by-page against every wide table and the architecture diagram.

### 4. Configuration reference

Environment variables read by the server ( `server/src/main.rs` ):

| Variable        | Default                                                                                                          | Purpose                                                                                                |
|-----------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| PORT            | 8080<br><small>SigniQ · Team Prometheus · SIH26141 · classical software simulation — no quantum hardware</small> | Listen port                                                                                            |
| HOST            | loopback                                                                                                         | Bind address ( 0.0.0.0 for LAN — deliberate act)                                                       |
| FRONTEND_DIST   | ../frontend/dist                                                                                                 | Static dashboard directory                                                                             |
| AUDIT_LOG       | audit_log.jsonl                                                                                                  | Audit ledger path                                                                                      |
| QDS_EVENT_LOG   | qds_events.jsonl                                                                                                 | QDS event log path; <b>its sibling &lt;name&gt;.replay-state.json holds the temporal-trap snapshot</b> |
| USERS_FILE      | users.json                                                                                                       | Account store (atomic tmp+rename writes)                                                               |
| TRENT_SEED      | unset                                                                                                            | Pin the QDS notary's setup randomness (deterministic demos)                                            |
| DEVELOPER_TOKEN | unset                                                                                                            | Optional development bypass token                                                                      |

**Isolated instance pattern** (used by the smoke test; never touches real data):

```
PORT=8090 AUDIT_LOG=target/scratch/audit.jsonl QDS_EVENT_LOG=target/scratch/qds.jsonl \
USERS_FILE=target/scratch/users.json target/debug/server.exe
```

Data files are created by the server in its working directory; runtime JSONL/JSON files are local state and belong in `.gitignore`.

## 5. API reference

Fifty-three routes ( `server/src` ). Bearer tokens come from `POST /api/auth/login` . Document endpoints generally accept an **optional** bearer token — unauthenticated calls operate on the shared session user; authenticated ones are per-user workspaces. Verified per-route in the isolated smoke test ( `scripts/smoke-isolated.mjs` ).

### System & auth

| Method & path                            | Purpose                     |
|------------------------------------------|-----------------------------|
| GET /api/health                          | Liveness probe              |
| GET /api/server-info                     | Build/server metadata       |
| POST /api/auth/register · login · logout | Account lifecycle           |
| GET /api/auth/me · users                 | Session identity; user list |

### QKD simulation

| Method & path | Purpose                                                                                                                                                                                                                                 |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST /api/run | Six-state QKD run (scenarios: secure, attack, sweep, blind-challenge...); returns per-scenario results + CHSH Bell test + statistical bounds + security accounting. Distilled secrets become the caller's session key (never returned). |

| Method & path                                                                                                   | Purpose                     |
|-----------------------------------------------------------------------------------------------------------------|-----------------------------|
| POST /api/simulate<br>SigniQ · Team Prometheus · SIH26141 · classical software simulation — no quantum hardware | Channel simulation variants |
| GET /api/events · GET /api/doc/events                                                                           | SSE progress streams        |

## QDS

| Method & path                       | Purpose                                                                                                    |
|-------------------------------------|------------------------------------------------------------------------------------------------------------|
| POST /api/qds/setup                 | Configure notary parameters ( $q, \lambda$ )                                                               |
| POST /api/qds/sign · verify         | Sign message / verify signature (temporal-trap enforced)                                                   |
| GET /api/qds/attacks                | Six-class attack battery (all must reject)                                                                 |
| GET /api/qds/forgery-analysis       | Empirical vs theoretical forgery probability                                                               |
| GET /api/qds/metrics? trials=&seed= | Confusion matrices, per-class detection, false-accept/false-alarm rates, timings                           |
| GET /api/qds/events                 | QDS event log                                                                                              |
| POST /api/qds/consensus-ring        | m-receiver consensus with k-of-m gates                                                                     |
| POST /api/doc/qds/key               | Derive the caller's document-sealing key from a six-state QDS session (commitment returned, never the key) |

## Documents & transfer

| Method & path                                                                          | Purpose                                                      |
|----------------------------------------------------------------------------------------|--------------------------------------------------------------|
| GET /api/doc/session · GET /api/doc/quorum                                             | Session/quorum state                                         |
| POST /api/doc/seal · verify · open                                                     | Seal file → .qsig ; verify container; open (quorum-gated)    |
| POST /api/doc/quorum/distribute · pledge · unlock                                      | Shamir k-of-m workflow                                       |
| POST /api/doc/send · receive                                                           | Peer delivery (same-server user or LAN peer URL)             |
| GET /api/doc/inbox · outbox · DELETE /api/doc/inbox/{id} · DELETE /api/doc/outbox/{id} | Mailboxes                                                    |
| POST /api/doc/inbox/verify                                                             | Cryptographic verification of an inbound container           |
| POST /api/doc/transfer                                                                 | Streaming transfer (SSE narration)                           |
| POST /api/doc/attack                                                                   | Tamper battery: tamper_bytes / swap_meta / reseal / truncate |
| POST /api/doc/attack-theater                                                           | Staged capture → tamper → forward → victim-rejection replay  |

| Method & path                                                                                                                      | Purpose                                     |
|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| POST /api/doc/ring/send · ring/attest<br>SigniQ · Team Prometheus · SIH26141 · classical software simulation — no quantum hardware | Consensus-ring delivery + attestation tally |
| POST /api/doc/relay/deposit · claim · GET<br>/api/doc/relay/inbox                                                                  | Relay-drop workflow                         |
| GET /api/doc/wire-proof                                                                                                            | Wire-level proof artifact                   |

## Audit & statistics

| Method & path                            | Purpose                                                                              |
|------------------------------------------|--------------------------------------------------------------------------------------|
| GET /api/audit/events?limit=             | Ledger entries + root                                                                |
| GET /api/audit/verify                    | Full chain verification                                                              |
| GET /api/audit/root                      | Current Merkle root                                                                  |
| GET /api/audit/proof?seq=                | Inclusion proof for one entry                                                        |
| GET /api/audit/export                    | Portable self-contained bundle (≤ 10,000 entries)                                    |
| POST /api/audit/clear                    | Wipe ledger — <b>destructive, operator-only</b>                                      |
| GET /api/stats/bounds                    | Chernoff–Hoeffding confidence engine (intervals, threshold curves, verdict p-values) |
| POST /api/blind-challenge/start · reveal | Blind statistical challenge demo                                                     |

## 6. Data formats

### 6.1 Audit ledger entry ( audit/src/lib.rs::AuditEntry )

JSONL, one entry per line, append-only:

| Field        | Meaning                                                                  |
|--------------|--------------------------------------------------------------------------|
| seq          | 1-based sequence number                                                  |
| ts           | RFC3339-ish UTC timestamp                                                |
| kind         | seal / verify / flag / transfer / quorum / attack / keygen / auth / open |
| label        | Actor/action, e.g. alice→bob                                             |
| accepted     | false = a threat or rejection was recorded                               |
| detail       | Human-readable line                                                      |
| payload_hash | SHA-256 of the primary artifact (binds log to document/signature)        |
| leaf_hash    | Chain link including previous leaf                                       |
| hash_v       | Leaf encoding version (2 = length-prefixed; absent = legacy)             |

Leaves are domain-separated ( `AUD1` prefix, length-framed fields); Merkle interior nodes carry the RFC 6962 `0x01` tag (second-preimage defense). `GET /api/audit/verify` re-walks the chain; inclusion proofs let any single entry be verified against the root.

[High against the root](#) · [SIH26141](#) · classical software simulation — no quantum hardware

## 6.2 .qsig container ( `sealing/src/lib.rs` )

```
magic      "QSIG1"
meta       { name, size, mime, sha256, sealed_at }
audit_ref { first_seq, last_seq, root }      - Merkle coverage of the sealing events
payload    { v, scheme, key_commitment, nonce, tag, quorum, ciphertext }
```

- **v2 payload:** AES-256-GCM under the canonical session key; container metadata bound as GCM associated data; independent HMAC-SHA256 tag; 12-byte nonce from a fresh 64-bit per-seal nonce.
- **v3 envelope:** the container JSON itself is GCM-sealed — on disk only the header, key commitment (a digest), and envelope nonce are visible.
- **Quorum mode:** seal key Shamir-split over GF(251); k shares reconstruct; below k it is cryptographically unrecoverable.
- Key commitment = `SHA-256(canonical_key)` with the final byte reduced into GF(251) — one definition shared by sealing, server, and quorum reconstruction.

## 6.3 Temporal-trap snapshot

`<qds_event_log>.replay-state.json` — JSON snapshot of the notary's consumed nonces, sifting-window sequence, and chain head, rewritten atomically (tmp+rename) after every mutation and restored at boot. A restart must never re-arm a consumed signature; the smoke test asserts the notary clock stays monotone across signatures.

## 7. Frontend architecture

- **Stack:** React 19 + TypeScript + Vite 8; plain CSS design system ( `src/index.css` , "Nocturne"); lint via `oxlint`.
- **Panels** ( `src/components/` ): `LiveMonitor` (SSE progress, charts capped at 240 points, 80 ms throttling), `QdsLab` , `DocVault` , `AttackLab` , `TransferPortal` , `PeerTransfer` , `LedgerPanel` (8 s poll + `__ledgerRefresh` hook), `SweepPanel` , `StatusCard` , `ConsensusRing` , `BlindChallenge` , `NoiseVsEvePanel` , `Constellation` , `Rite` , `TeleportAnim` , `GuidedRun` (guided tour), `Team` (dossier), `Charts` (lazy-loaded Recharts split — keeps the main bundle under the chunk warning).
- **API client:** `src/api.ts` . **Offline evidence:** `src/auditEvidence.ts` builds a self-contained verifier page from an audit export and verifies bundle-internal consistency in the browser.
- **Backgrounds** are canvas-based and visibility/reduced-motion aware.

## 8. Testing strategy

| Layer       | Location                                                                                                                                                                                                                 | What it proves                                                                                                                                                                                                                  |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unit tests  | inline per crate ( <code>qds</code> teleport physics, <code>detection</code> bounds, <code>pa</code> extractor collisions, <code>sealing</code> GCM/quorum, <code>audit</code> chain/Merkle, <code>temporal</code> trap) | Each mechanism against its paper's predicted statistics                                                                                                                                                                         |
| Integration | <code>tests/</code> crate                                                                                                                                                                                                | Cross-crate protocol flows                                                                                                                                                                                                      |
| Live smoke  | <code>scripts/smoke-isolated.mjs</code>                                                                                                                                                                                  | 47 end-to-end checks against a scratch instance (\$4 pattern): auth, key distillation, seal/verify, 4-mode tamper rejection, ring $k-1/k$ gates, 6-class QDS battery, metrics, bounds, ledger integrity, export, attack theater |
| Frontend    | <code>npm run build</code> + <code>npm run lint</code>                                                                                                                                                                   | Type-clean, lint-clean production bundle                                                                                                                                                                                        |

The older `scripts/e2e-*.mjs` scripts drive three-laptop scenarios but **create persistent accounts** — run them only against disposable instances.

The main binary ( `cargo run -p main_app` ) is a fifth verification layer: one seeded run that exercises every crate end-to-end and prints the Chernoff–Hoeffding dossier, the seal/tamper/verify round-trip, and the evaluation table — useful as a quick wiring check after refactors.

## 9. Troubleshooting

| Symptom                                                                                 | Cause & fix                                                                                                                                                                |
|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cargo build -p server</code> fails with "Access is denied (os error 5)" (Windows) | A running server is executing <code>target/debug/server.exe</code> and pins the file. Kill the process ( <code>taskkill //PID &lt;pid&gt; //F</code> ), rebuild, relaunch. |
| Dashboard shows 404 for assets                                                          | <code>frontend/dist</code> missing or stale — run <code>npm run build</code> in <code>frontend/</code> , or point <code>FRONTEND_DIST</code> at the built directory.       |
| Port already in use                                                                     | Another instance holds <code>PORT</code> ; pick a new port or stop the old instance ( <code>netstat -ano</code> )                                                          |
| Ledger restored "N malformed lines"                                                     | A truncated final line (kill during write). The chain verifier reports status; entries before the truncation remain intact.                                                |
| <code>replay-state restore failed</code> (starting fresh)                               | Corrupt snapshot — the notary deliberately starts fresh rather than refusing to boot; consumed signatures from before the corruption may be re-verifiable.                 |
| Frontend fetches fail from a different origin                                           | Use the server-origin page, or configure a proxy ( <code>frontend/vercel.json</code> holds a placeholder for split deployments).                                           |
| Metrics endpoint slow with high <code>trials</code>                                     | CPU-bound Monte-Carlo; it runs off the async reactor ( <code>spawn_blocking</code> ) — reduce <code>trials</code> (clamped 50–2000).                                       |



## 10. Operator security notes

---

- The server binds loopback by default; `HOST=0.0.0.0` exposes it to the network — an intentional act requiring firewall thought. Deployment configuration (Dockerfile, Render Blueprint) is covered in the repository README's *Deploying* section.
- Bearer tokens authenticate workspaces; document endpoints tolerate unauthenticated calls as the *shared* session user for demo convenience — do not expose that posture beyond a trusted LAN.
- Raw session keys and seal keys never leave the server; clients see commitments.
- `POST /api/audit/clear` is destructive and unauthenticated in the demo posture — restrict it before any shared deployment.
- The audit ledger proves internal consistency only; publisher authentication requires an independently trusted root. There is no key-revocation lifecycle.